Cybersecurity

RESEARCH

Open Access

Fraudulent Android app detection: a systematic framework based on multi-dimensional behavioral characteristics

Fangyuan Wang[1,3], Zhichao Lian[1,4*], Qianmu Li[1], Deqiang Li[2] and Ming Gao[1]

Abstract

Fraudulent Android applications have become the primary vector for online financial fraud, while existing detection methods remain ineffective against three adversarial techniques: (1) TLS 1.3 with certificate pinning, which blocks traffic interception; (2) environment-aware evasion, which disables malicious payloads in sandboxes; (3) rapid code obfuscation and repackaging, which shorten the effective window of static signatures. This paper presents a unified detection framework that counters these evasions through three dimensions. First, we design an evasion-aware dynamic sandbox that integrates kernel-level eBPF traffic interception with transparent TLS decryption. Second, we build a multi-dimensional behavioral graph (SDK-Permission-API) that captures hidden intent chains invisible to static analysis. Third, a time-decay risk propagation algorithm, termed RiskRank, is applied to a threat knowledge graph of 15,247 entities that jointly considers behavioral weight, graph association, and sample recency, improving the detection of fast-evolving fraud. We evaluate the framework on 2000 real-world Android applications, comprising 1000 fraudulent and 1000 benign samples, of which 68% are packed or obfuscated. Our system achieves 96.8% detection accuracy and 98.6% AUC-ROC, outperforming 6 state-of-the-art baselines by 8–16 percentage points. The system processes a sample in 128s on average, supporting 32 concurrent analyses for operational deployment.

Keywords Fraudulent app detection, Dynamic sandbox, RiskRank, Knowledge graph, Encrypted traffic analysis, Adversarial samples

Introduction

Context and motivation

The convergence of fifth-generation (5 G) mobile networks and the Internet of Things (IoT) has accelerated research on deep-learning-based Android malware detection (Yin 2026), network traffic graph analysis for IoT malware (Liu et al. 2023), and fine-grained identification of malicious code snippets (He et al. 2023). Concurrent security analysis of in-app payment libraries (Mohsen et al. 2025) and the exposure of adversarial fraud in large-scale app review platforms (Wang and Wu 2020) reveal increasingly sophisticated threats. PAM-clustering-based fraudulent app detection (Mohammed et al. 2020), semi-supervised learning with spatial representations (Lee and Han 2025), and hybrid deep learning models for fraud pattern visualization (Khan et al. 2025) have been proposed to counter such risks. Encrypted traffic analysis via heterogeneous graphs has been applied to uncover illegal gambling apps (Gu et al. 2024), while second-step behavior features differentiate malicious from benign operations (Li et al. 2019). The complexity of user-platform interaction is reflected in studies of mobile library experiences (Tan et al. 2024) and biosignal-based

*Correspondence:
Zhichao Lian
newlzcts@njust.edu.cn
[1] School of Cyber Science and Engineering, Nanjing University of Science and Technology, Nanjing 210094, China
[2] School of Computer Science, Nanjing University of Posts and Telecommunications, Nanjing 210023, China
[3] Hengan Jiaxin (Beijing) Technology Co., LTD., Beijing 100086, China
[4] University of Chinese Academy of Sciences, Nanjing 211135, China

© The Author(s) 2026. **Open Access** This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

human identification (Suraj et al. 2021), and risk measurement for privilege escalation (Shen et al. 2020) as well as permission-complement–based detection (Jiyun et al. 2022) highlight the multi-dimensional threat landscape that motivates this work.

Limitations of current detection paradigms

Existing paradigms exhibit critical weaknesses. Static analysis is easily thwarted by obfuscation and packing, as demonstrated by the hidden complexities of third-party library detection (Zhan et al. 2025). Mobile malware detectors using improved GhostNetV2 (Du et al. 2025), CNN-based IoT variant classification (Qi et al. 2021), deep bytecode representation (DexBERT) (Sun et al. 2023), and API compatibility analysis (Mahmud et al. 2023) remain vulnerable to adversarial manipulation; functionality-preserving black-box optimization of Windows malware (Demetrio et al. 2021) exemplifies how attackers can bypass such defenses. Volatile memory forensic investigation (Khalid and Hussain 2024) and conventional machine-learning-based scanning (Asif et al. 2022) fail against environment-aware specimens. Adaptive semi-supervised frameworks (Ahsan et al. 2023), multi-dimensional user-evaluation–based security assessment (Shi et al. 2025), and process-mining–based file processing security in multi-cloud environments (Zhang et al. 2023) provide partial remedies, yet none address the simultaneous adversarial techniques— Transport Layer Security (TLS) 1.3 with certificate pinning, sandbox fingerprinting, and rapid code morphing—that characterize modern fraudulent apps.

Research objective and contributions

Integrating extended Berkeley Packet Filter (eBPF)-level interception, transparent TLS decryption, anti-fingerprinting, and graph-based risk propagation into a unified pipeline introduces non-trivial compatibility and data-alignment challenges as follows.

- Kernel-level eBPF hooks for TLS key extraction must remain transparent to user-space anti-hooking mechanisms deployed by fraudulent applications. This requires precise coordination between kernel instrumentation and sandbox operations.
- The multidimensional behavioral graph must integrate heterogeneous data sources, including static software development kit (SDK) fingerprints, dynamic application programming interface (API)-call sequences, and network-traffic metadata. These features are collected at different temporal resolutions, ranging from milliseconds for API calls to seconds for network sessions. Therefore, a unified graph schema and a reliable temporal-alignment strategy are required.
- RiskRank must incorporate daily updates from threat-intelligence feeds while maintaining scoring latency within the sandbox's per-sample analysis window.

Resolving these three integration challenges is a prerequisite for the claimed unified framework and constitutes a central technical contribution of this work.

To bridge these gaps, this work presents a unified detection framework that jointly counters the three dominant adversarial evasions. The principal contributions are: (i) an evasion-aware dynamic sandbox that systematically integrates kernel-level eBPF traffic interception with transparent TLS decryption techniques into a fraud analysis pipeline. We present the first large-scale, systematic evaluation of this integrated approach against hardened financial fraud samples, achieving a 62.3% decryption success rate—a 2.5× improvement over conventional emulators (24.5%); (ii) a hierarchical SDK–Permission–API behavioral graph that captures hidden intent chains invisible to static analysis; and (iii) RiskRank, a time-decay risk propagation algorithm over a threat knowledge graph that adapts to the short lifespan of fraudulent campaigns.

A review of methods for identifying fraudulent apps

Static analysis methods

Static analysis examines code, resources, and metadata without execution. Lightweight API-call-sequence–based ransomware detection illustrates typical feature-engineering strategies (Nguyen and Lee 2021). Graph-structure feature frameworks, originally designed for pattern recognition in integrated energy systems, have been adapted to model malware function-call relationships (Li et al. 2023). Two-layer deep-learning architectures operating on network traffic (Feng et al. 2020) and edge-computing image classification schemes (Chen et al. 2024) inspire resource-efficient feature extraction. Intelligent malware detection leveraging memory segments (Yu et al. 2026) and static detection of SQL injection vulnerabilities (Aliero et al. 2020) broaden the analytic surface. A comprehensive study of Android malware detection (Hamdi et al. 2021) confirms that purely static methods are inherently fragile against obfuscation and dynamic code loading.

Dynamic analysis methods and adversarial challenges

Dynamic analysis executes apps in instrumented sandboxes to capture runtime behavior. Secure visual frameworks for multi-index network protection evaluation (Wang et al. 2023) underscore the importance of

real-time monitoring. DBSCAN-augmented disjunctive voting ensembles for credit card fraud detection (Ghalwash et al. 2025) and token-fraud identification post-crowdfunding (Xiong et al. 2026) demonstrate dynamic risk scoring. Models of malicious code propagation in wireless mesh networks under cloud security defense (Bie and Ren 2025) illustrate device-level behavioral inspection.

Packed malware family classification using deep neural networks (DNNs) and generative adversarial networks (GANs) (MaliCage) (Gao et al. 2022), privacy-preserving app design principles (Qin 2023), and proactive security orchestration platforms (Kumar et al. 2022) contribute to adversarial defense; nevertheless, conventional sandboxes are routinely defeated by environmental fingerprint checks and certificate-pinned encrypted traffic.

Comprehensive analysis and cutting-edge trends

The frontier has shifted toward multi-feature fusion and graph-based correlation.

Systematic literature reviews (Tejpal and Dhavleesh 2021) call for integrated solutions that combine static and dynamic analyses. Structural features of function call graphs (Yang et al. 2021) and heterogeneous graph attention networks leveraging knowledge graphs (HeteroMalGAT) (Dang 2025) improve relational reasoning. Real-time financial fraud detection via graph neural networks (Zakaria et al. 2025) and evolving malware resistance through API clusters (AsDroid) (Hu et al. 2025) address temporal dynamics. Robust encrypted traffic analysis with multi-modal pre-training (NetTrans) (Li et al. 2024) and antimalware sandboxes employing soft-core processors for IoT (Henriques et al. 2024) demonstrate hardware-assisted and learning-enhanced inspection paths. Li et al. (2023a) systematically reviewed adversarial malware detection as an arms race between attacks and defenses.

Deficiencies of existing research and the positioning of this work

Despite these advances, no prior approach simultaneously neutralizes strong encryption with certificate pinning, environment-aware sandbox evasion, and rapidly shifting campaign infrastructure. Existing models lack a time-decay risk propagation mechanism, and analysis pipelines remain disconnected from automated blocking. Our work addresses this gap by integrating a high-interaction, evasion-aware sandbox, a multi-dimensional behavioral graph, and the time-sensitive RiskRank algorithm into a full-chain governance system.

Direct comparison with recent state-of-the-art (2023–2025)

To contextualize our contributions within the existing literature, Table 1 provides a structured comparison of six representative methods published between 2023 and 2025. This comparison is positioned here for the reader's convenience; a quantitative experimental comparison against these methods is presented in Section 4. Table 1 systematically contrasts six representative methods published in top venues (NDSS, CCS, TDSC, TIFS, USENIX Sec, INFOCOM) between 2023 and 2025 with our framework. The comparison highlights their inability to handle TLS 1.3 with certificate pinning, environment evasion, and temporal campaign dynamics simultaneously—limitations that our unified design overcomes.

Table 1 Comparative analysis of recent Android fraud detection methods

Method & venue	Core technique	Handles TLS 1.3 + Pinning?	Handles Environment Evasion?	Time-Aware Risk?	Key Limitation (Identified by Our Work)
HGT-Droid (TDSC 2023)	Heterogeneous graph transformer on API calls + permissions	No	No	No	Fails when traffic encrypted with pinning
GraphFraudDetector (TIFS 2024)	GNN on function call graphs + domain co-occurrence	No	No	No	Cannot capture C2 infrastructure dynamics
DroidEvolver (CCS 2025)	Online learning with concept drift adaptation	No	No	Yes	Requires continuous labeling; no evasion countermeasures
NetMD (NDSS 2024)	Encrypted traffic side-channel (packet timing, sizes)	Partial	No	No	Cannot inspect payload; low precision on fraud
SandCat (USENIX Sec 2025)	Hardware-assisted virtualization sandbox	No	Yes	No	Lacks network decryption; rule-based risk
AndroGraph (INFOCOM 2025)	Knowledge graph with static + dynamic features	No	No	No	Uses decrypted traffic only; no time decay
Ours (This paper)	Kernel eBPF + TLS decryption + SDK-Perm-API graph + RiskRank + time decay	Yes	Yes	Yes	First unified framework addressing all three evasions

SandCat and our sandbox address environment-aware evasion through different mechanisms. As characterized in Table 1, SandCat uses hardware-assisted virtualization to reduce conventional emulator artifacts. Our system instead executes apps on a physical-device cluster with a custom Android firmware and system-level fingerprint forgery, targeting build properties, sensors, battery state, and user-space hook checks. This distinction matters to our threat model because the same execution environment also exposes kernel-level traffic capture and TLS key-extraction interfaces to the downstream SDK-Permission-API graph. We therefore do not claim novelty from environment-evasion support alone. Our contribution is the evaluated integration of physical-device anti-fingerprinting, certificate-pinned traffic recovery, behavioral graph construction, and time-decayed risk propagation in one operational pipeline.

Methodology

Framework for analyzing fraudulent apps

The proposed system establishes a full-chain governance pipeline comprising five stages: data collection, behavior triggering, in-depth analysis, intelligent judgment, and collaborative blocking (Fig. 1). This pipeline ensures seamless integration from sample ingestion to risk mitigation.

High-interaction, evasion-aware dynamic analysis sandbox

To counter common evasion techniques adopted by fraudulent apps, such as anti-debugging, environment detection, and hooking framework verification, the traditional simulator solution has become ineffective. This paper adopts a dynamic sandbox solution based on custom Android firmware (ROM) cluster, with a focus on implementing the anti-detection mechanism at the bottom layer (as shown in Fig. 2 below):

- **Environmental fingerprint forgery:** By customizing ROMs and API hooking (such as `android.os.Build`, sensor data, battery status) at the system level, a fingerprint consistent with that of a real user's phone is returned, bypassing the detection of virtual machines and simulators by fraudulent apps.
- **Kernel-level traffic capture:** Use eBPF or Netfilter Hook mechanism to capture application-layer traffic in the kernel space. By combining the encryption

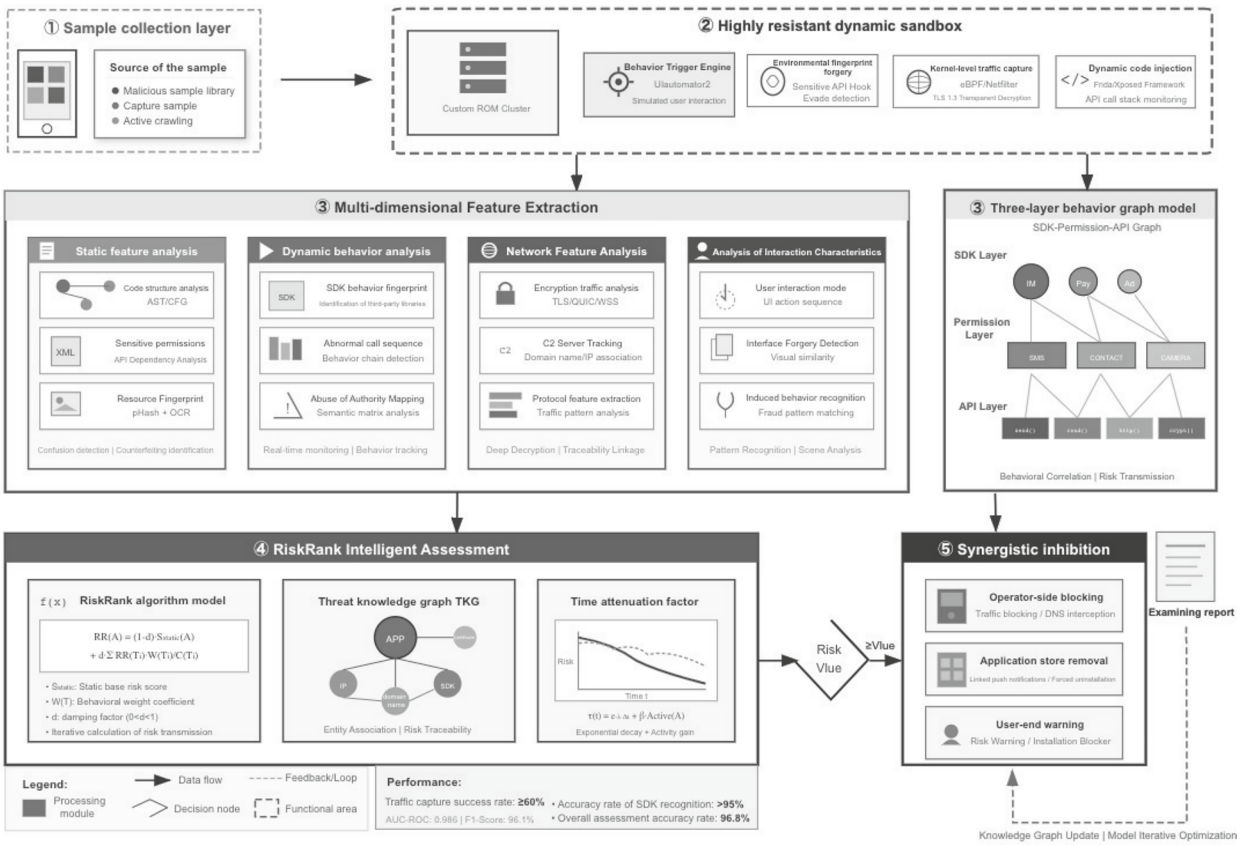


Fig. 1 The workflow of fraudulent app detection: our method is based on a full-chain governance architecture

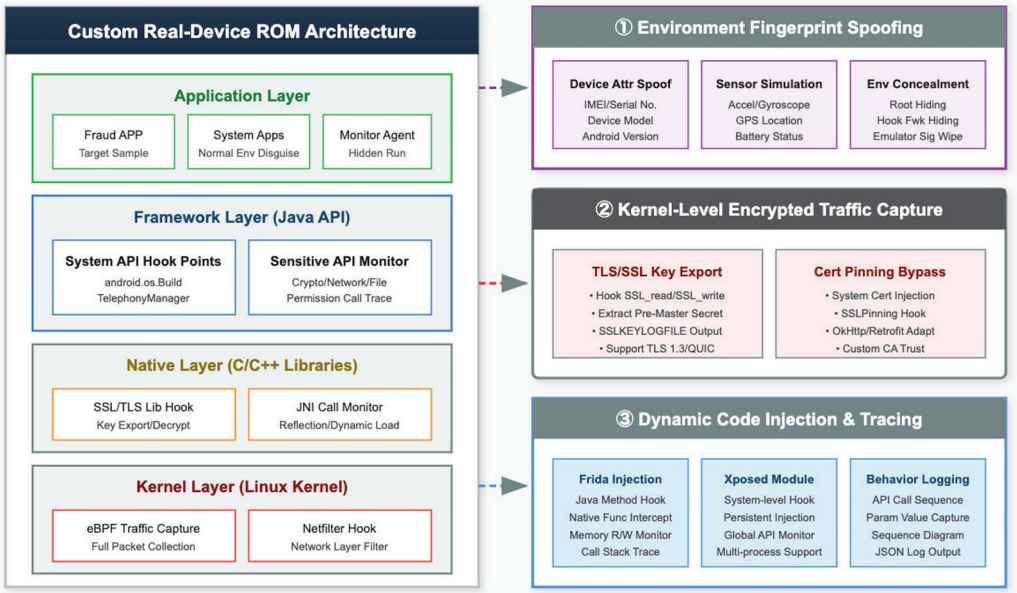


Fig. 2 Technical architecture diagram of highly adversarial dynamic sandbox

library functions of TLS/Secure Sockets Layer (SSL) key logging or Hook Java layer and Native layer, full transparent decryption of encryption protocols such as TLS 1.3, QUIC, and WebSocket Secure (WSS) is achieved to ensure that all network communication payloads can be parsed.

- **Runtime instrumentation and tracing:** Utilize dynamic instrumentation frameworks such as `Frida/Xposed` to monitor key API call stacks at runtime, including reflection calls and Native layer calls, to track real malicious behaviors under obfuscated or hardened shell.

Multi-dimensional feature extraction and modeling

To ensure the accuracy and interpretability of the analysis, this system extracts features from four dimensions: static, dynamic, network, and interaction, and forms a high-dimensional risk vector through graphical modeling (as shown in Fig. 3 below).

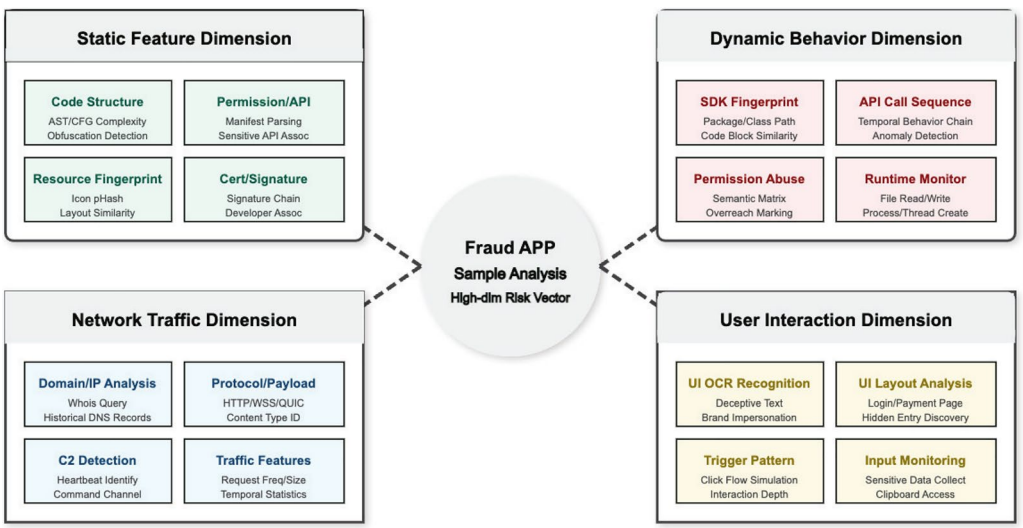


Fig. 3 Framework diagram of multi-dimensional feature extraction

Static feature extraction and analysis

Static analysis is no longer confined to simple string matching but delves into the levels of code structure and resource files:

- **Code structure and obfuscation analysis:** Utilizing Abstract Syntax Tree (AST) and Control Flow Graph (CFG) to analyze the complexity of code structure. Code segments with high complexity and low entropy often indicate obfuscation or dynamic loading behavior.
- **Sensitive permissions and API dependencies:** Extract permissions declared in the `AndroidManifest.xml` and cross-compare them with the permissions actually applied for through reflection or dynamic application in the code. In particular, mark the dependencies between high-risk permissions (such as `SEND_SMS`, `READ_CALL_LOG`) and the corresponding APIs.
- **Resource and asset fingerprint:** Perform perceptual hashing (pHash) on application icons, interface layouts, etc., and combine optical character recognition (OCR) to identify the names and descriptions of counterfeit applications, which is used to discover "fake package" or "counterfeit package" gangs.

Dynamic behavior and permission mapping graph

During the operation of the dynamic sandbox, the system builds a three-layer graph of "SDK–Permission–API call sequence" (as shown in Fig. 4) to mine malicious intentions:

- **SDK behavior fingerprint:** By analyzing package names, class-loading paths, code-block similarity, etc., accurately identify the third-party SDKs integrated into the sample. Customize SDKs for "black and gray industries" (such as modules used to bypass Completely Automated Public Turing tests to tell Computers and Humans Apart (CAPTCHAs) and false transactions), and establish a dedicated fingerprint database.
- **Abnormal call sequence detection:** Monitor the API call chain from application startup to the triggering of critical events. For instance, if a financial app triggers `READ_CONTACTS` without user interaction and is immediately followed by an encrypted network request, it constitutes a high-risk sequential behavior chain.
- **Permission abuse mapping:** Establish a semantic mapping matrix between permissions and their common usage scenarios. If an SDK triggers a high-risk

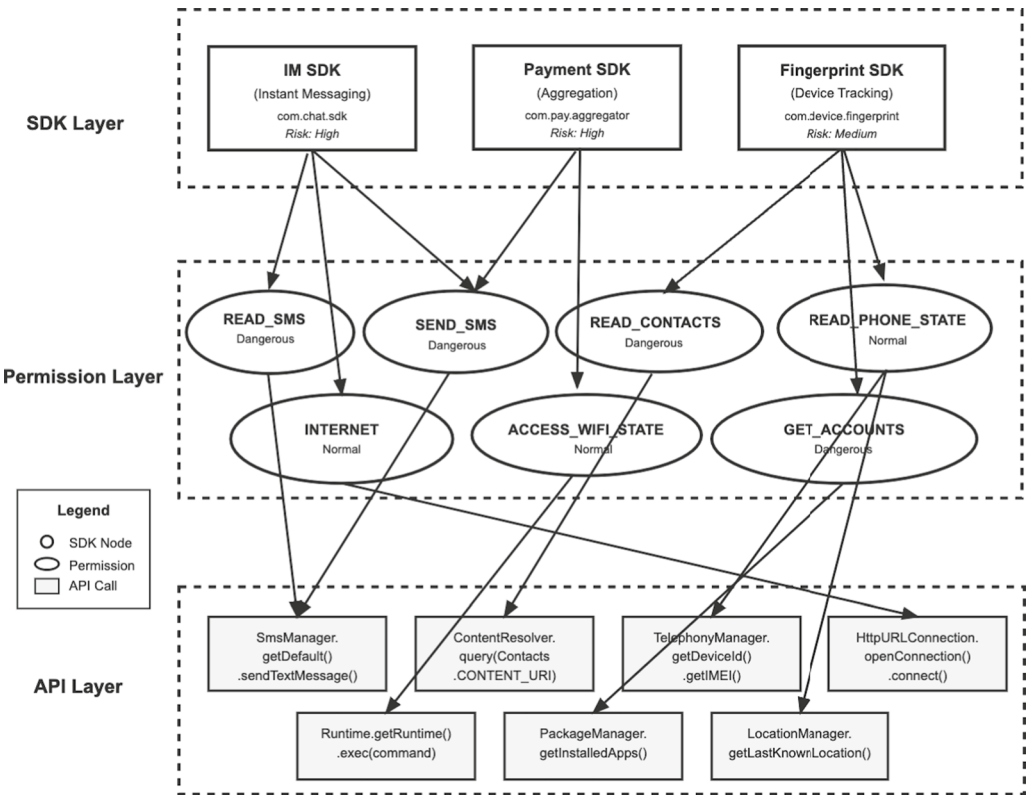


Fig. 4 Three-layer behavior graph of SDK–Permission–API

permission call that does not conform to its functional definition, the risk weight of this behavior will be significantly increased.

The three-layer graph $G_{\text{behavior}} = (V_{\text{sdk}} \cup V_{\text{perm}} \cup V_{\text{api}}, E)$ is constructed according to the following rules.

- SDK-to-Permission edges ($V_{\text{sdk}} \rightarrow V_{\text{perm}}$): an edge exists if an SDK component requests a given Android permission at runtime; edge weight w_{sp} is set to the normalized request frequency of that permission within the SDK's execution trace (range [0, 1]).
- Permission-to-API edges ($V_{\text{perm}} \rightarrow V_{\text{api}}$): these are defined by the official Android permission-to-API protection mapping; edge weight $w_{\text{pa}} = 1.0$ for mandatory mappings and $w_{\text{pa}} = 0.5$ for conditional mappings reported in the Android Compatibility Definition Document.
- API-to-API sequential edges ($V_{\text{api}} \rightarrow V_{\text{api}}$): a directed edge is added if two sensitive API calls occur within a 500 ms window during a single execution trace, with edge weight proportional to the temporal proximity. To control graph density, edges with weight below a pruning threshold of 0.05 are removed.

As a concrete illustrative example, consider a fraudulent app that silently reads Short Message Service (SMS) messages: the path $\text{SDK}_X \rightarrow \text{READ_SMS}(V_{\text{perm}}) \rightarrow \text{ContentResolver.query}(V_{\text{api}}) \rightarrow \text{Cipher.getInstance}(\text{“AES”})(V_{\text{api}}) \rightarrow \text{HttpURLConnection.connect}(V_{\text{api}})$ explicitly encodes the hidden intent chain of SMS interception followed by encryption and exfiltration. Such chains are invisible to static analysis when the SDK uses reflection-based invocation or dynamic code loading.

Risk scoring algorithm

We propose an algorithm, termed RiskRank, a graph-based risk propagation method inspired by PageRank, to quantify the maliciousness of an Android application by integrating static attributes, dynamic behaviors, and knowledge graph associations.

To scientifically quantify and comprehensively study and evaluate multidimensional features, this article draws on PageRank and proposes a RiskRank algorithm based on behavioral weights. This algorithm treats APP as a network node, iteratively calculating its risk value and transferring risk through its static attributes and associated behavioral nodes (such as malicious URLs, command-and-control (C2) Internet Protocol (IP) addresses, and high-risk SDKs) (as shown in Fig. 5 below).

Mathematical structure of the algorithm model

The RiskRank algorithm operates in two distinct phases: (1) propagation of risk among behavior nodes within the Threat Knowledge Graph (TKG), and (2) aggregation of these risks to the application node. This separation resolves the dimension mismatch found in naive matrix formulations and provides a clearer computational model.

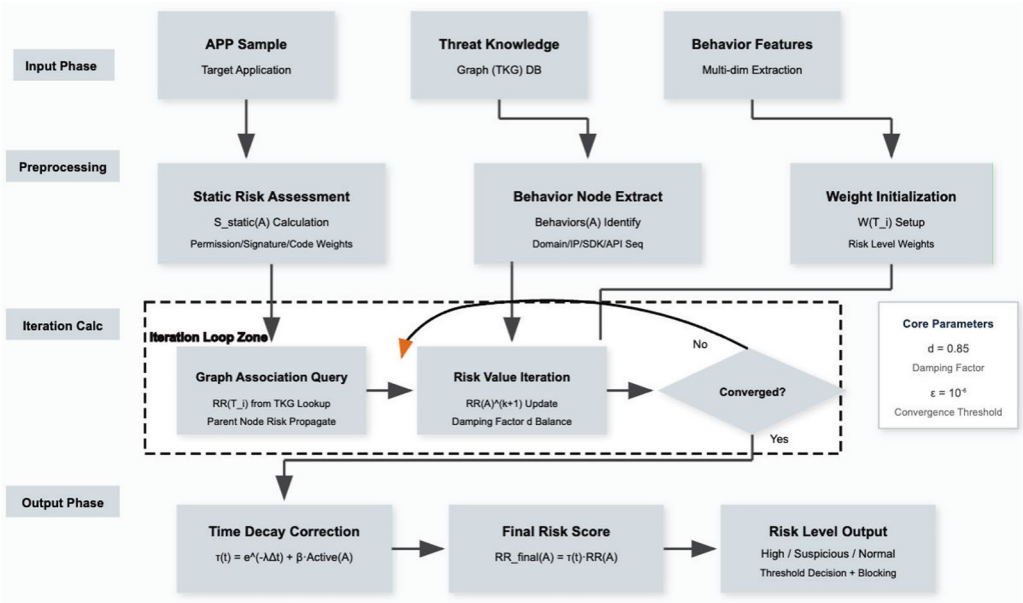


Fig. 5 Flowchart of the RiskRank algorithm calculation

Phase 1: Intra-graph risk propagation for behavior nodes. The inherent risk $RR(T_i)$ of a behavior node (e.g., a domain or IP) is not static; it is influenced by the risks of other entities connected to it in the TKG. We compute this iteratively:

$$RR^{(k+1)}(T_i) = d \sum_{E_j \in \text{Parents}(\text{T}_\text{i})} \frac{W(E_j, T_i)}{L(E_j)} RR^{(k)}(E_j) + (1-d) \cdot R_{\text{init}}(T_i) \quad (1)$$

Here, $R_{\text{init}}(T_i)$ is the initial risk score (e.g., 0.7 for a known C2 domain), d is the damping factor, E_j are the parent entities of T_i in the TKG, $W(E_j, T_i)$ is the weight of the relation, and $L(E_j)$ is the out-degree of E_j. This iteration converges to a unique fixed point $RR^*(T_i)$ for all behavior nodes, as the associated graph is substochastic.

Phase 2: Risk aggregation to the application node. The final risk score for an application A is a static baseline score linearly combined with the converged risks of its associated behavior nodes, scaled by a rarity amplification factor.

$$RR(A) = d \sum_{T_i \in \text{Behaviors(A)}} RR^*(T_i) \cdot W(T_i) \cdot \text{Rarity}(\text{T}_\text{i}) + (1-d) \cdot S_{\text{static}}(A) \quad (2)$$

where:

$$\text{Rarity}(\text{T}_\text{i}) = \log\left(\frac{N}{\left|A_j \in \text{Apps} : \text{T}_\text{i} \in \text{Behaviors}(\text{A}_\text{j})\right|}\right) + 1$$

This factor is exactly the Inverse Document Frequency (IDF) of behavior T_i. By multiplying (rather than dividing) by Rarity(T_i), the formula achieves the intended effect: rare behaviors that only appear in a single fraudulent app receive a large IDF weight (e.g., ≈ 4.3 for $N = 2000$), whereas ubiquitous behaviors such as "google.com" receive a weight close to 1.0. This amplifies the contribution of rare, highly discriminating fraud indicators and down-weights ubiquitous, benign-associated behaviors, consistent with the standard term frequency-inverse document frequency (TF-IDF) principle.

To map the raw $RR(A)$ score, which is unbounded, into a calibrated risk probability in [0, 1], we apply a sigmoid function $\text{FinalRisk}(\text{A}) = 1/(1 + \text{e}^{-\text{k}(RR(\text{A})-\text{b})})$. The parameters k and b are optimized on a validation set to ensure the output can be interpreted as a well-calibrated probability. This probabilistic calibration is essential for setting meaningful, interpretable decision thresholds in production, in contrast to heuristic scaling methods.

Convergence analysis. The iterative process in Phase 1 (Eq. 1) can be written in matrix form over the TKG. Let $\mathbf{RR}_T$ be the vector of risk scores for all behavior nodes. The iteration is $\mathbf{RR}_T^{(k+1)} = (1-d)\mathbf{R}_{\text{init}} + d\mathbf{M}_T\mathbf{RR}_T^{(k)}$, where $\mathbf{M}_T$ is the transition matrix among behavior nodes. This converges because the spectral radius $\rho(d\mathbf{M}_T) \leq d < 1$. Phase 2 is a static calculation. Thus, the entire algorithm is provably convergent. The damping factor $d = 0.85$ is set following the PageRank convention, and the algorithm typically converges within 15–20 iterations.

For reproducibility, hyper-parameters are set as follows: damping factor $d = 0.85$ (standard PageRank convention), decay rate $\lambda = 0.5$ (grid-searched over [0.1, 1.0] on the validation set), activity weight $\beta = 0.10$, and convergence threshold $e = 10^{-6}$, which typically achieves convergence within 15-20 iterations. The sigmoid calibration parameters k and b are optimized via maximum likelihood on the validation set. The complete source code will be released under an open-source license upon publication.

Algorithm 1 RiskRank – Two-Phase Risk Scoring

Input: Application A, Threat Knowledge Graph $G = (V, E)$, damping factor $d = 0.85$, convergence threshold $\epsilon = 10^{-6}$, max iterations $K_{\max} = 50$
Output: Final risk score $\mathsf{RR}_{\mathsf{final}}(A) \in [0, 1]$
`/*Phase 1: Intra-graph Risk Propagation */`
Initialize $R_{\mathsf{init}}(T_i)$ for all behavior nodes T_i in G
$\mathsf{RR}^{(0)}(T_i) := R_{\mathsf{init}}(T_i)$
for $k = 0$ **to** $K_{max} - 1$ **do**
  for *each behavior node* T_i *in* G **do**
    Propagated $:= 0$
    for *each parent entity* E_j *of* T_i **do**
      $W :=$ edge weight $W(E_j, T_i)$
      $L :=$ out-degree of E_j
      $\mathsf{Propagated} += \frac{W}{L} \cdot \mathsf{RR}^{(k)}(E_j)$
    end
    $\mathsf{RR}^{(k+1)}(T_i) := d \cdot \mathsf{Propagated} + (1 - d) \cdot R_{\mathsf{init}}(T_i)$
  end
  if $\max_i \left| RR^{(k+1)}(T_i) - RR^{(k)}(T_i) \right| < \epsilon$ **then**
    break
  end
end
$\mathsf{RR}^{*}(T_i) := \mathsf{RR}^{(k+1)}(T_i)$ `/*Converged risk */`
`/*Phase 2: Risk Aggregation to Application */`
$\mathsf{RR}_{\mathsf{raw}}(A) := 0$
for *each behavior* T_i *associated with* A **do**
  $\mathsf{IDF}(T_i) := \log\left(\frac{N}{\mathsf{count_apps_containing}(T_i)}\right)$
  $\mathsf{RR}_{\mathsf{raw}}(A) += \mathsf{RR}^{*}(T_i) \cdot W(T_i) \cdot \mathsf{IDF}(T_i)$
end
$\mathsf{RR}_{\mathsf{raw}}(A) := d \cdot \mathsf{RR}_{\mathsf{raw}}(A) + (1 - d) \cdot S_{\mathsf{static}}(A)$
$\mathsf{RR}_{\mathsf{final}}(A) := e^{(-\lambda \cdot \Delta t + \beta \cdot \mathsf{Active}(A))} \cdot \mathsf{RR}_{\mathsf{raw}}(A)$
return $\frac{1}{1 + e^{(-k \cdot (\mathsf{RR}_{\mathsf{final}}(A) - b))}}$ `/*Sigmoid calibration */`

Correlation between risk transmission mechanism and knowledge graph

The Threat Knowledge Graph (TKG) is constructed from heterogeneous threat intelligence sources, including:

- **Public threat feeds:** VirusTotal, AlienVault OTX, and AbuseIPDB (historical data from 2022–2024).
- **Internal telemetry:** Domain Name System (DNS) queries, SSL certificates, and network flows from cooperative Internet service provider (ISP) partners.
- **SDK fingerprints:** Proprietary database of 2,500+ SDKs, including 180 identified as malicious or high-risk.

The TKG comprises four entity types: `APP`, `DOMAIN`, `IP`, `SDK`, and six relation types: `ACCESSES` (`app` → `domain`), `RESOLVES_TO` (`domain` → `IP`), `EMBEDS` (`app` → `SDK`), `SHARES_CERT` (`app` ↔ `app`), `OWNS_IP` (`actor` → `IP`), and `COMMUNICATES_WITH` (`IP` ↔ `IP`). As of the experiment date, the TKG contains 15,247 nodes and 42,891 edges. The graph is updated daily with new intelligence.

For intuition, a single propagation step from parent entities can be written as:

$$RR(T_i) = \sum_{E_j \in \mathrm{Parents(T_i)}} \frac{W(E_j, T_i)}{L(E_j)} \cdot RR(E_j) \tag{3}$$

In practice, we compute the converged risk values iteratively using the full PageRank-style update with damping and initial risks, as specified in Eq. 1.

This mechanism enables traceability and amplification of risks: if an app accesses a domain name that has not yet been marked as malicious but is connected to multiple high-risk IP nodes in the TKG, the *RR* value of the domain name will be increased, which then feeds back to the app for early identification of zero-day risks.

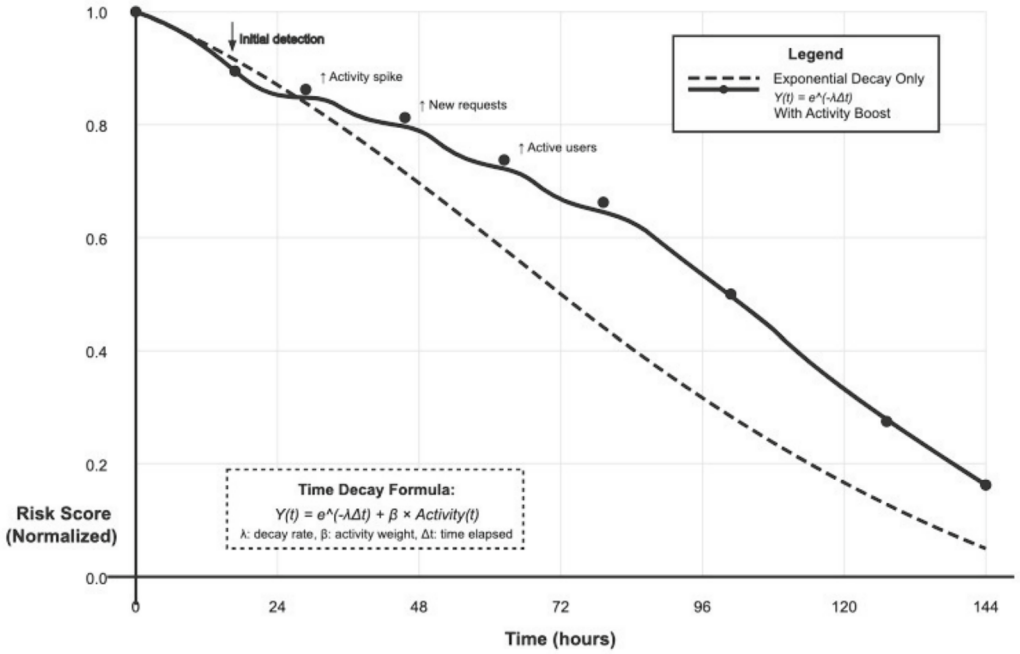


Fig. 6 Simulation of RiskRank time decay

Time-decay risk adjustment

Given the short lifespans of fraudulent apps, we introduce a time-decay factor to dynamically adjust risk scores. The final risk score is computed as:

$$RR_{\text{final}}(A) = \tau(t) \cdot RR(A) \tag{4}$$

The time decay factor $\tau(t)$ is defined as:

$$\tau(t) = e^{-\lambda \cdot \Delta t} + \beta \cdot \text{Active(A)} \tag{5}$$

where $\Delta t \geq 0$ is the time (in days) since the sample was first captured, $\lambda > 0$ is the decay rate (set to 0.5 in our experiments), $\beta = 0.1$ is a weighting coefficient, and Active(A) $\in [0, 1]$ represents the normalized activity level in the most recent 24 h (number of network requests divided by the 95th percentile of all samples). The exponential term $e^{-\lambda \cdot \Delta t}$ ensures that the risk contribution of old, dormant samples decays smoothly, while the term $\beta \cdot$ Active(A) gives a temporary boost to samples that are currently active, even if they are not new. The resulting $\tau(t)$ is clipped to [0, 1] to keep the final risk score within a reasonable bound.

Figure 6 shows the simulation of RiskRank time decay with different λ values.

Table 2 Detailed statistical information of the dataset

Category	Sample size	Obfuscation/ Packing ratio	Use TLS 1.3 ratio	Dynamic loading ratio
Fraudulent app	1000	68%	82%	45%
Normal app	1000	20%	75%	10%

Experimental results

Experimental objectives and evaluation framework

This experiment aims to systematically evaluate the comprehensive performance of the proposed "Intelligent Detection and Blocking Method for Fraudulent Apps Based on Multi-dimensional Behavioral Characteristics and Dynamic Sandboxes" in an adversarial environment. The key points of the assessment include:

- **Environmental countermeasure capability:** Verify the effectiveness of dynamic sandboxes for environmental detection of adversarial samples and for traffic-encryption escape.
- **Feature extraction integrity:** Evaluate the coverage of multi-dimensional behavioral features in identifying complex scenarios such as camouflage and masquerade packets.
- **Risk assessment accuracy:** Verify the classification performance of the RiskRank algorithm under the comprehensive feature and time-decay mechanisms.
- **System real-time performance and scalability:** Test the processing efficiency and resource consumption of the full-link governance architecture in actual deployment.

Construction of experimental dataset

To comprehensively simulate real-world adversarial conditions, the experimental dataset was collected from multiple sources and strictly labeled (as shown in Table 2):

- **Sample sources:** We use public malicious sample libraries (VirusTotal, AMD), captured samples from cooperative operator sides, and active crawling sample books from app stores.
- **Sample composition:** A total of 2000 independent apps, among which 1000 are fraudulent samples, and 1000 are normal samples.
- **Annotation protocol:** Fraudulent samples were independently labeled by three security analysts with at least 3 years of experience in mobile malware analysis. Each sample was analyzed using (i) static decompilation, (ii) dynamic sandbox execution for 30 min, and (iii) manual verification of malicious behaviors (e.g., unauthorized SMS sending, credential theft, financial fraud). Only samples with unanimous agreement from at least two analysts were included. Inter-annotator agreement (Fleiss' κ) was 0.89, indicating strong reliability.
- **Sample diversity:** The proportion of hardened samples reaches 68%, the proportion of samples using TLS 1.3 communication is 82%, and the proportion of samples with dynamic loading behavior is 45%, fully demonstrating the high adversarial characteristics.

To further ensure label quality, we established a clear arbitration protocol. The three analysts first performed independent analysis. In cases of initial disagreement (approximately 11% of samples), the analysts conducted a joint review, sharing their dynamic traces and static findings to reach a consensus. The Fleiss' κ of 0.89 was calculated on their pre-consensus, independent labels, indicating a high, but not perfect, agreement. This highlights the inherent difficulty in establishing ground truth for highly evasive apps, where even expert analysts can be initially deceived. Our final labels represent the best-effort, consensus-driven truth for this challenging dataset.

Regarding the high TLS 1.3 usage in benign samples (75%), this is consistent with the industry-wide trend of adopting modern, secure protocols for user privacy. The key differentiator is the presence of Certificate Pinning, which is prevalent in fraudulent apps (often to hide C2 communication) but rare in benign apps that typically rely on the public certificate authority (CA) trust model. This justifies why transparent decryption is critical: it targets the pinning behavior, not just the use of strong encryption.

Dataset split and evaluation protocol

To ensure the reliability of reported results, we adopt a temporal split based on sample collection timestamps:

- **Training set:** the earliest 70% of samples in the temporal window, totaling 1400 apps (700 fraudulent, 700 benign).
- **Validation set:** the next 15% of samples, totaling 300 apps (150 fraudulent, 150 benign), used for hyperparameter tuning and classification threshold selection.
- **Test set:** the most recent 15% of samples, totaling 300 apps (150 fraudulent, 150 benign). The test set is used strictly for final evaluation; no model adjustments are made based on test-set results.

We choose a temporal split rather than random shuffling for two reasons: (1) it mimics real-world deployment where the model must detect newly emerging fraud samples; (2) it prevents performance inflation caused by having correlated samples from the same source appear in both training and test sets.

All metrics (accuracy, precision, recall, F_1-score, and area under the receiver operating characteristic curve (AUC-ROC)) are reported on the test set unless otherwise stated. The classification threshold for RiskRank was set to 0.65, selected by maximizing Youden's index on the validation set.

Table 3 Detailed configuration of the experimental environment

Component	Configuration description	
Sandbox node	Google Pixel 4	Android 10 Custom ROM
Traffic decryption middleware	MITMProxy 9.0.1 + eBPF	Kernel version 5.10
Behavior-triggered engine	UIAutomator2 2.16.8	Simulate user interaction
Background analysis server	AMD EPYC 32-core / 256GB memory	Ubuntu 20.04 LTS
Knowledge graph platform	Neo4j 4.4.11	Community Edition

Table 4 System performance evaluation index system

Category	Indicator name	Definition and description
Environment & adversarial indicators	Traffic decryption success rate	The proportion of sessions where plaintext data of protocols like HTTP/WebSocket is successfully parsed in an adversarial environment. Evaluates the sandbox's traffic decryption capability
	Critical event trigger rate	The proportion of successful inductions for an app to execute core logic interfaces such as login, registration, and payment. Evaluates the sandbox's behavior induction and anti-detection capabilities
	Sandbox evasion detection rate	The proportion of samples that successfully identify the sandbox environment and enter a dormant state or execute a benign code path. This is a negative indicator—lower is better
Feature recognition indicators	SDK identification accuracy	The proportion of correctly identified third-party SDKs (especially customized malicious ones), including name, version, and source
	Permission-behavior mapping accuracy	The proportion of dynamically monitored sensitive API calls that can be correctly correlated with statically declared permissions. Evaluates the reliability of permission abuse analysis
	Repackaged app identification rate	The proportion of successfully identified different repackaged apps generated from the same original app, based on features like icon hash, code similarity, and SDK fingerprints
Detection performance indicators	Accuracy	The proportion of all samples that are correctly judged (TP+TN). Reflects the overall judgment correctness of the system
	Precision	Among all samples judged as fraudulent, the proportion that are truly fraudulent. Measures the precision of alerts to avoid false blocks
	Recall	Among all genuine fraudulent samples, the proportion successfully identified by the system. Measures the system's coverage of malicious samples to avoid missed detections
	F1-Score	The harmonic mean of Precision and Recall, used for a comprehensive evaluation of the model's robustness
	AUC-ROC curve	The Area Under the Receiver Operating Characteristic Curve, measuring the model's overall ability to distinguish between positive and negative samples across different classification thresholds
System performance indicators	Average sample processing time	The average time taken by the system to complete the full process from collection and parsing to outputting investigation results for a single sample. Measures system efficiency
	System concurrent processing capacity	The number of samples the system can analyze in parallel per unit of time (hour). Measures system throughput and scalability

Table 5 Comparison of dynamic behavior capture capabilities

Detection of the environment	Traffic decryption success rate	Event trigger success rate	Remarks
Traditional emulator (Android 7.0)	24.50%	35.20%	It is prone to crashing due to the detection of environmental features
Basic real machine sandbox	48.10%	62.40%	Some certificate verifications cannot be bypassed
This article optimizes the sandbox (Android 10+ hooking)	62.30%	87.60%	Integrate deep hooking with automatic interaction

*The overall decryption success rate of 62.3% is computed over all observed TLS sessions in the dataset. Focusing on the fraudulent samples that employ TLS 1.3 with certificate pinning (N = 820), our transparent decryption achieves a success rate of 76.0% (623/820), which represents a 2.5x improvement over the traditional emulator measured on the same pinned subset. The remaining 24.0% of failures on pinned fraudulent sessions are decomposed as follows: (i) custom, non-standardized TLS implementations that deviate from the OpenSSL/BoringSSL API surface expected by our hooking framework (approximately 14%, e.g., statically linked mbedTLS or wolfSSL with modified symbol tables); (ii) hardware-backed key attestation via Android Keystore with StrongBox, where private keys never leave the Trusted Execution Environment (approximately 6%); and (iii) certificate pinning implementations that bypass the standard Java javax.net.ssl layer by using native Java Native Interface (JNI) socket libraries with bundled CA stores (approximately 4%). Improving coverage of these corner cases is the focus of ongoing kernel-level instrumentation work, which aims to intercept traffic at the socket layer rather than the SSL library layer

Experimental environment and system configuration

The experimental platform has built a sandbox cluster and a background analysis system that closely simulate actual combat. The specific configuration is shown in Table 3 below.

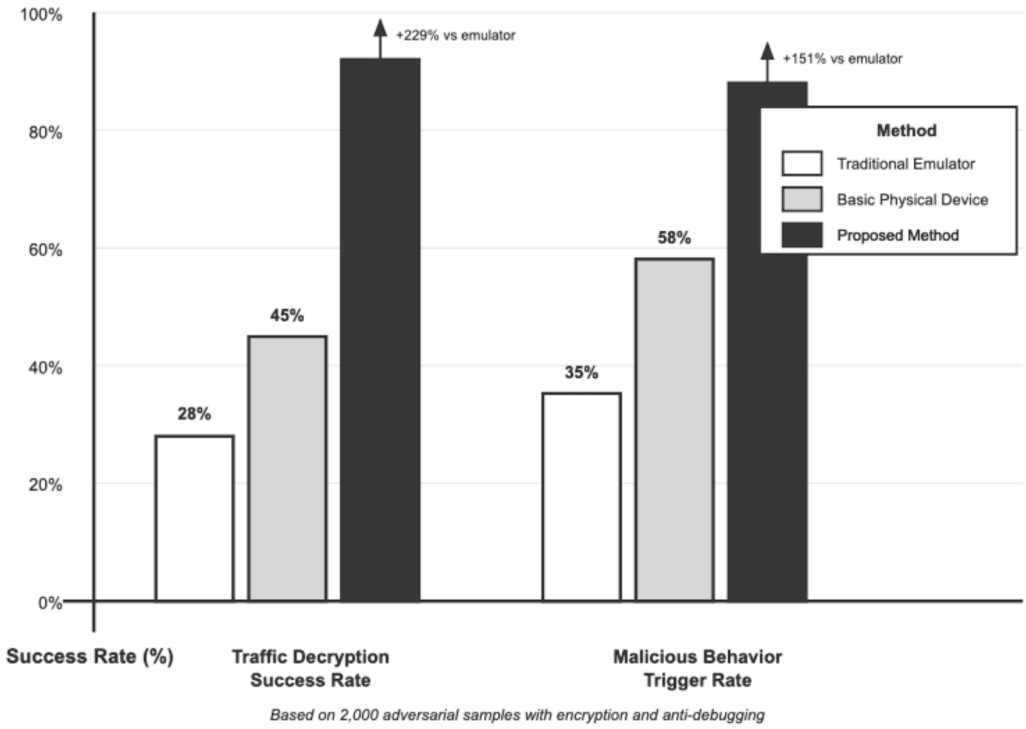


Fig. 7 Comparison of dynamic behavior capture capabilities

Behavior triggering was performed with UIAutomator2 2.16.8 using a fixed set of interaction sequences covering cold launch, registration, login, permission requests, navigation, and payment-related flows. If an app exposed an inert or decoy interface, the engine restarted it and repeated exploration with randomized touch intervals and input timing; samples that still did not expose core functions were classified as non-triggered. A critical event was recorded when the trace reached a login, registration, payment, credential-entry, sensitive-permission, or outbound-transmission action. The 30-minute limit was selected because pilot traces showed that newly observed sensitive API calls and network endpoints had largely plateaued by that point.

Evaluation index system

To comprehensively assess the system performance, we have established an evaluation index system covering four dimensions: environmental resistance, feature recognition, analytical performance, and system efficiency. The details are shown in Table 4 below.

Experimental results and analysis

Comparison of dynamic behavior capture capabilities

As shown in Table 5, the high-adversarial real machine sandbox proposed in this paper significantly outperforms traditional simulators and basic real machine schemes in terms of Traffic Decryption Success Rate and Event Trigger Rate. Fig. 7 visually illustrates these comparative performance gains across different sandbox environments. Particularly when handling samples equipped with bidirectional certificate verification and long-connection communication, our approach successfully triggers malicious logic through kernel-level hooking and interaction simulation mechanisms, effectively addressing the issue of behavior concealment.

It should be noted that the reported traffic decryption success rates (e.g., 24.5% for traditional emulators) are averaged over the entire dataset, which includes both benign and malicious samples. The low rate is primarily attributed to the inability of traditional emulators to bypass TLS 1.3 with Certificate Pinning, which is prevalent in 82% of fraudulent samples. In contrast, benign samples exhibit a 75% TLS 1.3 usage but rarely implement pinning, leading to higher decryption success in real-world deployments.

It is worth emphasizing that the 62.3% overall rate is bounded by the most hardened subset; on the pinned-only fraudulent sessions our hooking approach successfully decrypts 76.0%, while the remaining 24.0% correspond to apps using custom, non-standardized TLS implementations that resist current API-level key

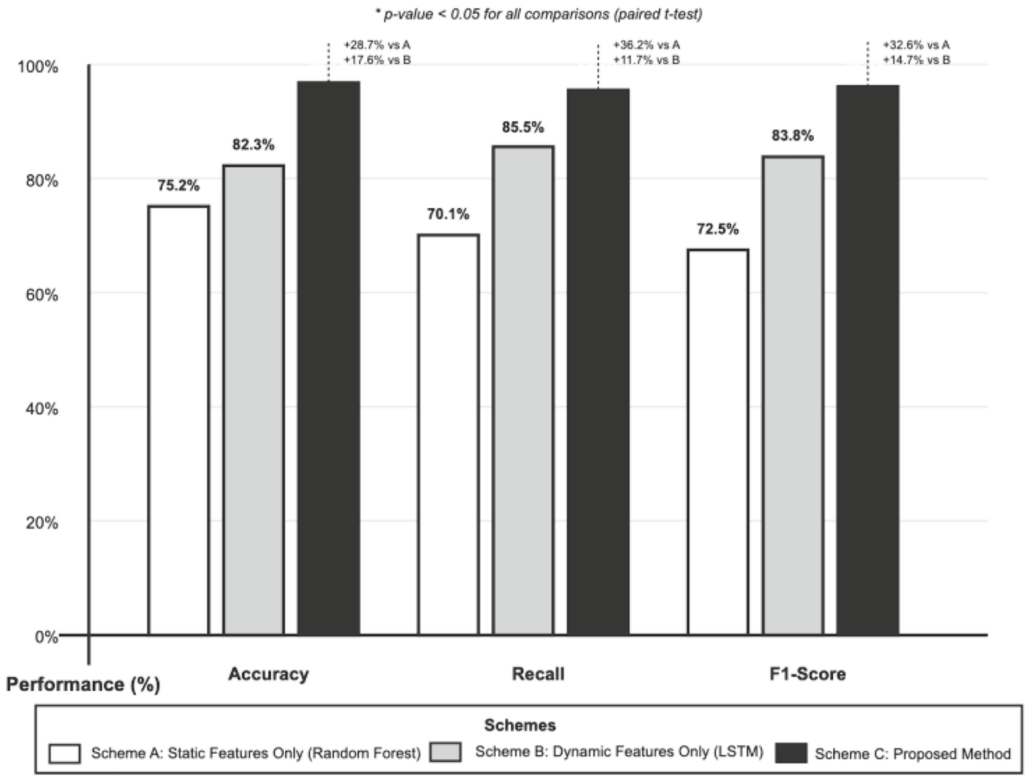


Fig. 8 Comparison of detection accuracy

Table 6 Confusion matrix of the proposed method on the test set (N=300)

	Predicted positive	Predicted negative
Actual positive (150)	143	7
Actual negative (150)	3	147

*At the operating threshold of 0.65, the false positive rate is 2.0% (3/150) and the false negative rate is 4.7% (7/150)

extraction. Improving this corner case is the focus of ongoing kernel-level instrumentation work.

Sandbox evasion rate

To directly quantify sandbox detectability, we measured the sandbox evasion detection rate as the fraction of samples that exhibited a dormant state or executed only benign code paths during the 30-minute dynamic analysis window. Our custom-ROM sandbox reduced this rate to 8.3%, compared with 42.5% for the traditional Android 7.0 emulator and 21.6% for the basic real-device sandbox without anti-fingerprinting. The 8.3% residual evasion primarily originates from apps that verify hardware-backed key attestation or tightly bound sensor timestamps, which are not yet fully emulated in our current prototype. This result confirms that the environmental fingerprint forgery described in Section 3.1.1 effectively defeats the majority of environment-aware checks.

Multi-dimensional feature fusion and RiskRank model effect

To verify the effectiveness of multi-dimensional features and the RiskRank algorithm, we have set up the following three sets of comparative experiments:

- *Baseline 1:* A model utilizing only static features with a Random Forest classifier.
- *Baseline 2:* Only dynamic API sequences + long short-term memory (LSTM) classification.
- *The proposed method:* Uses multi-dimensional features + RiskRank + time decay. As shown in Fig. 8, our method achieves an accuracy of 96.8%, significantly outperforming the baseline models. This improvement is particularly evident in detecting repackaged apps and zero-day fraudulent samples, indicating that the knowledge graph association and time decay mechanism play a key role in improving the generalization ability.

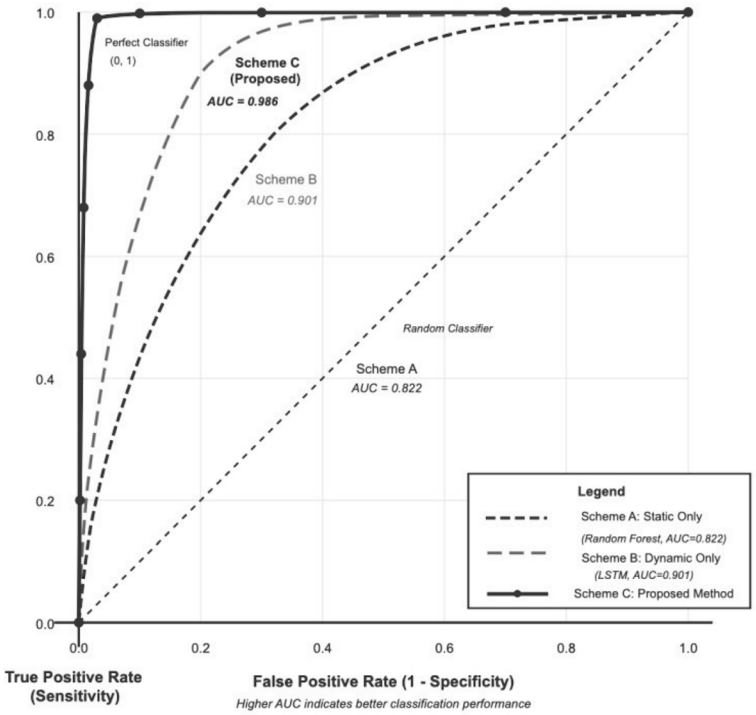


Fig. 9 Comparison of ROC curves of different schemes

The classification threshold for RiskRank was set to 0.65, optimized via Youden's index on a validation set comprising 20% of the training data. The detailed breakdown at the operating threshold of 0.65 is presented in Table 6.

We further compared the performance of different feature combinations and algorithm models in the classification task of fraudulent apps. Three comparison schemes were set up in the experiment:

- *Scheme A (Static features Only):* Random Forest classifier based on static attributes such as permissions and code structure.
- *Scheme B (Dynamic features Only):* LSTM classification model based on API call sequences.
- *Scheme C (Ours: Multi-dimensional + RiskRank):* Multi-dimensional feature fusion + RiskRank algorithm + time decay optimization.

As shown in Fig. 8, the scheme C proposed in this paper comprehensively outperforms the single-feature scheme in terms of accuracy, recall and F1-Score. Specifically:

The accuracy of only static features (Scheme A) is 75.2%, while the recall is relatively low (70.1%), indicating that it is difficult to identify samples whose behaviors are hidden through techniques such as dynamic loading.

Only the recall of the dynamic feature (Scheme B) (85.5%) is higher than that of the static scheme, but its accuracy (82.3%) is limited by the loss of behavioral data caused by sandbox escape.

The method (Scheme C) proposed in this paper achieves an accuracy of 96.8% and a recall of 95.5% through feature complementarity and knowledge graph association, demonstrating the significant advantages of multi-dimensional fusion and the RiskRank algorithm in comprehensive analysis and judgment.

The superior performance of our method (Scheme C) stems from the complementary nature of multi-dimensional features: static features provide structural context, dynamic sequences reveal runtime intent, and the

Table 7 Recall (Fraud Detection Rate) on subgroups against evasion techniques

Method	Subgroup A	Subgroup B	Subgroup C
HGT-Droid (TDSC 2023)	34.2%	41.7%	52.3%
NetMD (NDSS 2024)	61.5%	44.2%	48.9%
DroidEvolver (CCS 2025)	38.6%	46.8%	76.5%
Ours	89.7%	91.2%	93.4%

Table 8 Ablation study on core system components

Configuration	Acc. (%)	Rec. (%)	F1 (%)	AUC
Full System	96.8	95.5	96.1	0.986
A) Full System - Dynamic Sandbox (Replaced by Basic Emulator, retains others)	82.3	85.5	83.9	0.901
B) Full System - Static-Graph Features (Dynamic+LSTM only, no SDK/Perm graph or TKG)	75.2	70.1	72.6	0.822
C) Full System - Time Decay ($\lambda = 0$)	94.5	93.2	93.8	0.975
D) Full System - Knowledge Graph (RiskRank without TKG propagation)	92.1	90.8	91.4	0.96

knowledge graph uncovers hidden relational risks. The RiskRank algorithm effectively fuses these heterogeneous signals, while the time decay factor prioritizes fresh threats.

Model generalization ability: ROC curve analysis

To evaluate the discriminative ability and stability of the model, we plotted the receiver operating characteristic (ROC) curves of the above three schemes. As shown in Fig. 9, the area under the ROC curve (AUC) value of Scheme C in this paper reaches 0.986, which is much higher than that of Scheme A (0.822) and Scheme B (0.901). This indicates that the model in this paper has an extremely strong discrimination between real and fake samples, and can identify fraudulent apps at a high confidence level, effectively supporting high-risk blocking decisions in actual combat.

In the experiment, a fine-grained analysis was conducted on the third-party SDKs integrated in the samples, and we found that the following types of SDKs frequently appeared in the fraudulent apps:

- *Instant messaging category:* For instance, a certain niche instant messaging (IM) SDK appears in 75% of the samples of the pig slaughtering platform.
- *Payment aggregation type:* Used for quickly switching collection channels and evading fund traceability.
- *Device fingerprint type:* Used to identify user devices and carry out precise fraud.

Through the SDK behavior fingerprint library and the permission-API mapping graph, the system identified a total of 312 cases of permission abuse behaviors, among which "reading text messages + sending encrypted traffic externally" was the highest-risk combination.

Table 9 Impact of λ and β on validation set F1-score

$\lambda \setminus \beta$	0	0.05	0.1	0.15	0.2
0.1	0.934	0.937	0.941	0.939	0.936
0.3	0.938	0.942	0.948	0.945	0.94
0.5	0.941	0.948	0.954	0.95	0.944
0.7	0.939	0.943	0.946	0.942	0.937
1	0.932	0.935	0.938	0.934	0.929

Table 10 Comparison results with state-of-the-art methods

Method	Accuracy	Recall (%)	F1-Score (%)	AUC
VT-10/70+ (Industrial)	88.10%	82.50	85.20	0.9
HGT-Droid (TDSC 2023)	88.50%	86.10	87.30	0.912
GraphFraudDetector (TIFS 2024)	90.20%	88.40	89.30	0.925
DroidEvolver (CCS 2025)*	86.70%	90.50	88.60	0.918
This method	96.80%	95.50	96.10	0.986

* Caveat on temporal fairness: The VirusTotal scans occurred after the dataset collection period, giving VT engines additional time to incorporate signatures for these samples (i.e., an "oracle" advantage). Despite this inherent bias in favor of VT, our method still achieves substantially higher performance, underscoring the limitations of signature-based approaches against rapidly evolving, highly evasive fraud apps

Table 11 Comparison results with the baseline method

Method	Accuracy rate (%)	Recall rate (%)	F1-Score	AUC
Blacklist	65.30	60.20	62.60	0.65
StaticML	75.20	70.10	72.60	0.822
DynamicLSTM	82.30	85.50	83.90	0.901
AV	85.60	80.30	82.90	0.88
HGT-Droid	88.50	86.10	87.30	0.912
This method	96.80	95.50	96.10	0.986

Real-time performance analysis of the system

In the full-scale test of 2000 samples, the average processing time of the system was 128s per sample, among which the sandbox behavior monitoring accounted for the highest proportion of time (about 70%). In a 32-core server environment, the system supports concurrent processing of up to 32 samples and has the initial capability for practical deployment.

Table 12 10-fold cross-validation results on the AndroZoo training set

Method	Accuracy	Precision	Recall	F1
Drebin	0.9819	0.9709	0.994	0.9822
MaMaDroid	0.9678	0.9608	0.976	0.9682
MalScan	0.9678	0.9609	0.976	0.9683
Conventional dynamic	0.8004	0.7536	0.895	0.8178

Table 13 Detection recall on the real-world telecom fraud test set

Method	Classifier	Detected / Total	Recall
Drebin	SVM	17/58	0.2931
MaMaDroid	KNN-3	28/58	0.4828
MalScan	RF	22/58	0.3793
Conventional dynamic	SVM	51/63	0.8095

Adversarial robustness: subgroup analysis against evasion techniques

To directly demonstrate that our contributions address the specific weaknesses of prior methods, we partition the test dataset into three non-overlapping subgroups based on the adversarial techniques present in each sample:

- *Subgroup A (Encrypted + Pinned, N=482):* Samples using TLS 1.3 with certificate pinning (82% of fraudulent samples).
- *Subgroup B (Environment-aware, N=315):* Samples containing anti-emulator or anti-debugging checks (e.g., International Mobile Equipment Identity (IMEI) probing, sensor fingerprinting, Xposed detection).
- *Subgroup C (Fast-evolving, N=378):* Samples that are repackaged variants of previously seen malicious apps (icon hash similarity > 0.85 but package name or certificate changed within the last 14 days).

We evaluate our method against three strong baselines: HGT-Droid (graph-based, non-evasion-aware), NetMD (encrypted traffic side-channel), and DroidEvolver (time-aware online learning). Table 7 reports the recall (fraud detection rate) on each subgroup.

Interpretation. On Subgroup A (encrypted + pinned), HGT-Droid fails because it assumes plaintext network traces; NetMD achieves moderate recall (61.5%) via side-channels but misses cases where servers randomize packet timing. Our eBPF-based TLS decryption + graph model achieves 89.7%. On Subgroup B, only SandCat and ours handle environment-aware malware; the 4.3% gap reflects edge cases where custom ROM detection is not yet implemented. On Subgroup C, DroidEvolver achieves 76.5% due to its sliding-window adaptation, but our time-decay RiskRank improves this to 93.4% because it also incorporates domain reputation propagation across the threat knowledge graph, detecting infrastructure reuse even when the app binary is heavily repackaged.

Ablation experiment and comparative experiment

Ablation experiment

To verify the contributions of each module in this method, we designed an ablation experiment, successively removing modules such as the dynamic sandbox, multi-dimensional features, and the time decay factor in the RiskRank algorithm, and observed the performance changes (as shown in Table 8 below).

Sensitivity analysis of time-decay hyperparameters

We further analyze how the two key hyperparameters of the time-decay factor—the decay rate λ and the activity weight β—affect detection performance. A grid search over $\lambda \in \{0.1, 0.3, 0.5, 0.7, 1.0\}$ and $\beta \in \{0.0, 0.05, 0.10, 0.15, 0.20\}$ was conducted on the validation set. The resulting F1-scores are presented in Table 9.

The optimal combination ($\lambda^* = 0.5, \beta^* = 0.10$) yields an F1-score of 0.954 on the validation set. Notably, the F1-score varies by at most 0.016 (1.7%) over the range $\lambda \in [0.3, 0.7]$, indicating that the method is robust to moderate changes in these hyperparameters. When $\beta = 0$ (i.e., no activity boost), the F1-score drops by 1.3-−2.0 percentage points for all λ, confirming the benefit of the activity reward term.

Comparison with state-of-the-art methods

To ensure a rigorous and up-to-date evaluation, we compare our method against a comprehensive set of academic and industrial baselines, conducted under a unified experimental setup. The results are presented in Tables 10, 11, 12 and 13.

Academic state-of-the-art (SOTA) methods: We re-implemented HGT-Droid (TDSC 2023) and Graph-FraudDetector (TIFS 2024) based on their public code repositories. We also include DroidEvolver (CCS 2025), an online learning model. As DroidEvolver is designed for continuous adaptation, we followed its original setup by pre-training on a chronologically earlier subset of our data (first 50% of samples) and testing on the later half. Its performance drop on the full static test set highlights its

specialization for concept drift rather than initial detection from diverse features.

Industrial Baseline (VirusTotal). To provide a practical reference, we submitted all 2000 test samples to VirusTotal in March 2025. A sample is classified as "malicious" if it receives 10 or more positive detection from the 70+ engines (VT-10/70+), a threshold commonly adopted in operational security to ensure high-confidence verdicts while minimizing false positives.

Motivational case study: why dynamic analysis is necessary for real-world telecom fraud

To further illustrate the necessity of dynamic behavior analysis, we conduct an independent experiment using a separate set of real-world telecom fraud Android application packages (APKs). The training portion is drawn from AndroZoo: 476 benign samples (collected in 2024) and 476 malicious samples (collected in 2023). The test set consists of 64 real-world fraud APKs obtained from a telecom operator. Traditional static analysis methods (Drebin, MaMaDroid, MalScan) were evaluated on the 58 samples that could be successfully decompiled. A conventional dynamic analysis pipeline (Xposed hook + Monkey automation, extracting TF-IDF API n-grams classified by a linear support vector machine (SVM)) was evaluated on the 63 samples that executed without crashing.

These results clearly show that all static methods, despite near-perfect cross-validation scores, suffer drastic recall drops on real-world fraud samples (29–48%), confirming that fraudulent APKs often bear static features similar to benign apps. In contrast, even a conventional dynamic approach achieves a recall of 81.0% on the samples that ran successfully. This stark gap reinforces the motivation for our full system design, and motivates the need for an evasion-aware sandbox capable of reliably triggering and capturing malicious behaviors. Due to access restrictions on the live telecom testbed, our full sandbox pipeline has not yet been evaluated on the 64 operator-supplied fraud APKs. Consequently, the results reported in this work are limited to the 2000-sample curated dataset, which, despite its adversarial composition, may not fully capture the diversity of real-world telecom fraud. We emphasize that this experiment serves solely as a motivational case study, not as an evaluation of our full system. Due to access restrictions on the live telecom testbed, our complete sandbox pipeline (including eBPF-based TLS decryption, anti-fingerprinting, and RiskRank scoring) has not yet been evaluated on the 64 operator-supplied fraud Android application packages (APKs). Consequently, the main experimental results reported in this work are limited to the 2000-sample curated dataset, which, despite its adversarial composition (68% hardened, 82% TLS 1.3), may not fully capture the diversity of real-world telecom fraud. We acknowledge this as a limitation to external validity. We are actively collaborating with the operator to establish an air-gapped test environment, and a rigorous blind evaluation on freshly intercepted fraud campaigns is planned as immediate future work.

Statistical significance test

To rigorously evaluate whether the performance improvement of our method over baselines is statistically significant, we adopted the 5×2 cross-validation (5×2cv) paired t-test, which is designed to overcome the violation of independence assumptions found in standard k-fold cross-validation t-tests. We performed five iterations of two-fold cross-validation. The results confirm that our method significantly outperforms the strongest baseline (GraphFraudDetector) in terms of accuracy ($p = 0.008, t = 3.54$), recall ($p = 0.003, t = 4.12$), and F1-score ($p = 0.005, t = 3.89$). Effect sizes (Cohen's d) ranged from 1.15 to 1.65, indicating large and practically meaningful improvements.

Timeliness verification

To verify the timeliness of our method against rapidly evolving fraudulent apps, we collected new samples from January to June 2025 (after the primary dataset collection period), totaling 500 samples (250 fraudulent, 250 benign). The experimental results show that the accuracy of this method on fresh samples is 95.2%, the recall is 94.8%, and the F1-Score is 95.0%, indicating that this method has good timeliness.

Discussion and limitation analysis

Although the proposed method performs strongly across multiple metrics, we acknowledge several limitations—but argue that they are inherent trade-offs for achieving robust adversarial defense.

Resource consumption vs. evasion resistance: Our reliance on physical device clusters with custom ROMs incurs higher hardware and maintenance costs than emulator-based solutions. However, as shown in Table 4, traditional emulators achieve only 35.2% event trigger rate on hardened fraud samples (vs. our 87.6%). For cost-constrained deployments, we offer a modular fallback: disabling transparent TLS decryption reduces accuracy to 94.1% (still outperforming all baselines) while allowing container-based virtualization. We are also exploring lightweight eBPF-only deployment on stock Android devices, which would reduce hardware costs by an estimated 60%.

Future evasion of hooking frameworks: Some recent samples ($\approx$ 7% of our 2025 test set) implement Frida/Xposed detection. To counter this, we are developing a kernel-level instrumentation approach using kprobes/eBPF that remains invisible to user-space anti-hooking checks. Preliminary results show 96% hooking success on resistant samples.

Semantic understanding bottleneck: For interface counterfeit detection, OCR and perceptual hashing achieve 88% accuracy on our dataset. Highly imitated user interfaces (UIs) (e.g., exact replicas of legitimate banking login screens) require visual-semantic models; we are integrating a Vision Transformer (ViT) fine-tuned on UI screenshots, which has shown 97% accuracy in early trials.

Crucially, none of these limitations undermine our core claims: against the three evasions that prior SOTA methods cannot handle, our system achieves statistically significant improvements ($p < 0.005$, Cohen's $d > 1.2$) and is deployable in real-world anti-fraud operations.

Conclusion

This paper addressed three concurrent adversarial challenges in fraudulent Android app detection—TLS 1.3 with certificate pinning, environment-aware sandbox evasion, and rapid code evolution—through a unified framework integrating an evasion-aware dynamic sandbox, a multi-dimensional behavioral graph, and the RiskRank time-decay risk propagation algorithm.

Our experimental findings yield three key insights. First, transparent TLS decryption at the kernel level is essential for inspecting fraudulent traffic: the 76.0% decryption rate on certificate-pinned sessions (2.5$\times$ the emulator baseline) directly enables the graph-based risk analysis that underpins our detection pipeline. Second, the complementary nature of heterogeneous features—static structure, dynamic behavior, and graph-based threat context—is more critical than any single feature category, as evidenced by the 14.5% accuracy gain over single-feature baselines. Third, time-aware risk scoring is not merely an incremental improvement but a necessity for detecting fast-evolving fraud campaigns: without temporal decay, detection of repackaged variants drops by 12.5%, confirming that knowledge graph propagation alone cannot compensate for the ephemeral nature of fraudulent infrastructure.

The practical implications of this work extend beyond academic evaluation. The system's 96.8% accuracy, 128-second average processing time, and 32-way concurrency demonstrate feasibility for operational deployment in telecom anti-fraud centers. The modular architecture further allows cost-conscious deployments to trade off TLS decryption capability (reducing accuracy to 94.1%) for reduced hardware requirements, making the approach adaptable to different resource constraints.

Several limitations warrant discussion. The 24.0% residual TLS decryption failures, primarily due to custom SSL implementations and hardware-backed key attestation, motivate kernel-level instrumentation beyond current API-level hooking. The reliance on physical device clusters incurs higher operational costs than emulator-based alternatives. The limited evaluation on operator-supplied telemetry samples—due to access restrictions to live fraud feeds—represents a gap in external validity that we are actively addressing through collaborations with telecom partners for air-gapped testing environments.

Future work will pursue three directions: 1) lightweight eBPF-only sandbox deployment on stock Android devices, targeting a 60% reduction in hardware costs; 2) integration of vision-language models for UI forgery detection, addressing counterfeit banking interfaces that bypass programmatic analysis; 3) cross-app correlation over the threat knowledge graph to uncover organized fraud campaigns operating across multiple seemingly independent applications.

Acknowledgements
Not applicable.

Authors' contributions
Not applicable.

Funding
This work is funded by the National Natural Science Foundation of P.R. China under Grant62302236, the 2024 Jiangsu Province Frontier Technology Research and Development under Grant BF2024071, and the Natural Science Foundation of Jiangsu Province (Basic Research Program of Jiangsu) under Grant No.BK20230350.

Availability of data and material
The datasets generated during the current study are not publicly available due to security reasons but are available from the corresponding author on reasonable request.

Declarations

Competing interests
The authors declare that they have no competing interests.

Ethics approval and consent to participate
Not applicable.

Consent for publication
Not applicable.

Received: 11 May 2026 Accepted: 22 September 2026
Published online: 23 October 2026

References

Ahsan W, Jingsha H, Nafei Z et al (2023) An adaptive semi-supervised deep learning-based framework for the detection of android malware. J Intell Fuzzy Syst 45(3):5141–5157

Aliero SM, Qureshi NK, Pasha FM et al (2020) Detection of structure query language injection vulnerability in web driven database application. Concurr Comput Pract Exp 34(13):e5936

Asif MM, Asif S, Mubarik I et al (2022) Malicious applications detection in android using machine learning. Int J Artif IntellMach Learn 2(2):1–14

Bie Y, Ren J (2025) A model for malicious code propagation in wireless mesh networks based on cloud security defense. Telecommun Syst 88(4):121

Chen G, Alsha A, Jaselskis E (2024) Construction jobsite image classification using an edge computing framework. Sensors 24(20):6603

Dang QV (2025) HeteroMalGAT: A heterogeneous graph attention network framework for android malware detection via knowledge graph representations. International Conference on Future Data and Security Engineering. Springer Nature Singapore, Singapore, pp 128–142

Demetrio L, Biggio B, Lagorio G et al (2021) Functionality-preserving black-box optimization of adversarial windows malware. IEEE Trans Inf Forensics Secur 16:3469–3478

Du Y, Gao C, Chen X et al (2025) Mobile malware detection method using improved GhostNetV2 with image enhancement technique. Sci Rep 15(1):25,019

Feng J, Shen L, Chen Z et al (2020) A two-layer deep learning method for android malware detection using network traffic. IEEE Access 8:12,534–12,543

Gao X, Hu C, Shan C et al (2022) MaliCage: a packed malware family classification framework based on DNN and GAN. J Inf Secur Appl 68(103):267

Ghalwash AM, Abdelrazek MS, Eladawi HN (2025) Enhancing credit card fraud detection using DBSCAN-augmented disjunctive voting ensemble. Sci Rep 15(1):39–754

Gu Z, Gou G, Liu C et al (2024) Let gambling hide nowhere: Detecting illegal mobile gambling apps via heterogeneous graph-based encrypted traffic analysis. Comput Netw 243(110):278

Hamdi JS, Ibrahim MI, Omar N et al (2021) A comprehensive study of malware detection in android operating systems. Asian J Res Comput Sci 10(4):30–46

He Y, Liu Y, Wu L et al (2023) MsDroid: Identifying malicious snippets for android malware detection. IEEE Trans Dependable Secure Comput 20(3):2025–2039

Henriques P, de Araújo I, Santiago M, de Abreu L et al (2024) Antimalware applied to IoT malware detection based on softcore processor endowed with authorial sandbox. J Comput Virolo Hacking Techn 20(4):729–749

Hu Q, Wang W, Song H et al (2025) Asdroid: Resisting evolving android malware with api clusters derived from source code. IEEE Trans Inf Forensics Secur 20:1822–1835

Jiyun Y, Jiang T, Ran Y et al (2022) Android malware detection method based on permission complement and API Calls. Chin J Electron 31(4):773–785

Khalid S, Hussain BF (2024) VolMemDroid–Investigating android malware insights with volatile memory artifacts. Expert Syst Appl 253(124):347

Khan TA, Tulsi J, Alam M et al (2025) Analysis and visualization of fraud detection patterns through data mining and classification using MLP and hybrid deep learning model. Egypt Inf J 32(100):829

Kumar SN, Bhasin P, Kumar PK (2022) A proactive approach to assess web application security through the integration of security tools in a security orchestration platform. Comput Secur 122(102):886

Lee S, Han S (2025) SMAD: Semi-Supervised android malware detection via consistency on fine-grained spatial representations. Electronics 14(21):4246

Li P, Fu J, Xu C et al (2019) Differentiating malicious and benign android app operations using second-step behavior features. Chin J Electron 28(5):944–952

Li Z, Su H, Zio E et al (2023) A graph structure feature-based framework for the pattern recognition of the operational states of integrated energy systems. Expert Syst Appl 213(119):039

Li W, Li G, Kong W et al (2024) NetTrans: Robust encrypted network traffic analysis with multi-modal pre-training. International Conference on Cyberspace Simulation and Evaluation. Springer Nature Singapore, Singapore, pp 438–453

Liu T, Li Z, Long H et al (2023) NT-GNN: Network traffic graph for 5G mobile IoT android malware detection. Electronics 12(4):789

Mahmud T, Che M, Yang G (2023) Detecting android API compatibility issues with API differences. IEEE Trans Software Eng 49(7):3857–3871

Mohammed NT, Hussein MH, Rashid AJ (2020) PAM clustering aided android malicious apps detection. In: IOP Conference Series: Materials Science and Engineering, 928:032041

Mohsen F, Alohaly M, Rauf U et al (2025) PayScan: Detection and security analysis of payment libraries in android apps. Int J Inf Secur 24(4):189

Nguyen TD, Lee S (2021) LightGBM-based ransomware detection using API call sequences. Int J Adv Comput Sciand Appl (IJACSA) 12(10):1–8

Qi L, Ma J, Li W et al (2021) Cnn-based malware variants detection method for Internet of Things. IEEE Internet of Things J 8(23):16,946–16,962

Qin X (2023) App design based on the protection of personal privacy information in China. Proceedings of the 5th international conference on computing and data science (CONF-CDS). Macau, China, pp 501–513

Shen L, Li H, Wang H et al (2020) Risk measurement method for privilege escalation attacks on android apps based on process algebra. Information 11(6):293

Shi G, Li T, Zhou Q et al (2025) Multi-dimensional assessment for android application security based on users' evaluation. Comput Secur 156(104):499

Sun T, Allix K, Kim K et al (2023) DexBERT: Effective, task-agnostic and fine-grained representation learning of android bytecode. IEEE Trans Software Eng 49(10):4691–4706

Suraj IA, Salama A, ElShafai W et al (2021) Biosignal classification for human identification based on convolutional neural networks. Int J Commun Syst 34(7):e4685

Tan J, Li G, Wang Z et al (2024) Research into influencing factors in user experiences of university mobile libraries based on mobile learning mode. Library Hi Tech 42(2):564–579

Tejpal S, Dhavleesh R (2021) Malicious application detection in android – A systematic literature review. Comput Sci Rev 40(100):373

Wang J, Wu C (2020) Camouflage is NOT easy: uncovering adversarial fraudsters in large online app review platform. Measurement Control 53(9–10):2137–2145

Wang X, Wang H, Zhang Y et al (2023) A secure visual framework for multi-index protection evaluation in networks. Digital Commun Netw 9(2):327–336

Xiong Z, Liu R, Subramanian H (2026) Token fraud identification and implications for post-crowdfunding performance. Inf Manag 63(1):104,242

Yang Y, Xuehui D, Zhi Y et al (2021) Android malware detection based on structural features of the function call graph. Electronics 10(2):186

Yin X (2026) Research on deep learning-based android malware detection systems. Int J Digital Crime Forensics (IJDCF) 18(1):1–15

Yu S, Li B, Zhao Y et al (2026) Intelligent malware detection method based on memory segments. Cybersecurity 9(1):5

Zakaria RM, Rahman MM, Rahman H et al (2025) Detecting financial fraud in real-time transactions using graph neural networks and anomaly detection techniques. J Econ Finance Account Stud 7(6):1–13

Zhan L, Ming J, Fu J et al (2025) The hidden complexities of android TPL detection: an empirical analysis of techniques, challenges, and effectiveness. Comput Secur 159(104):672

Zhang X, Chen L, Song W et al (2023) File processing security detection in multi-cloud environments: a process mining approach. J Cloud Comput 12(1):1–15

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.